Recursion

Introduction to computer programming

Management and Artificial Intelligence

Stacks

The **stack** is one of the most popular abstract data types in computer science.

A stack is a sequence (collection) of elements where only the "last" element (i.e., the one at the "top" of the stack) can be accessed.

The allowed operations on a stack $\mathcal{S}$ are the following:

- `push(` e `,` $\mathcal{S}$ `)`: adds a new element $e \in V$, for any domain V, at the top of the stack $\mathcal{S}$
- `pop(` $\mathcal{S}$ `)`: returns and removes the current element at the top of the stack $\mathcal{S}$.

Optionally, it is possible to define an additional operation:

- `peek(` $\mathcal{S}$ `)`: reads the top element without modifying the stack.

A special domain value is $\mathcal{S}_{\emptyset}$, namely the empty stack.

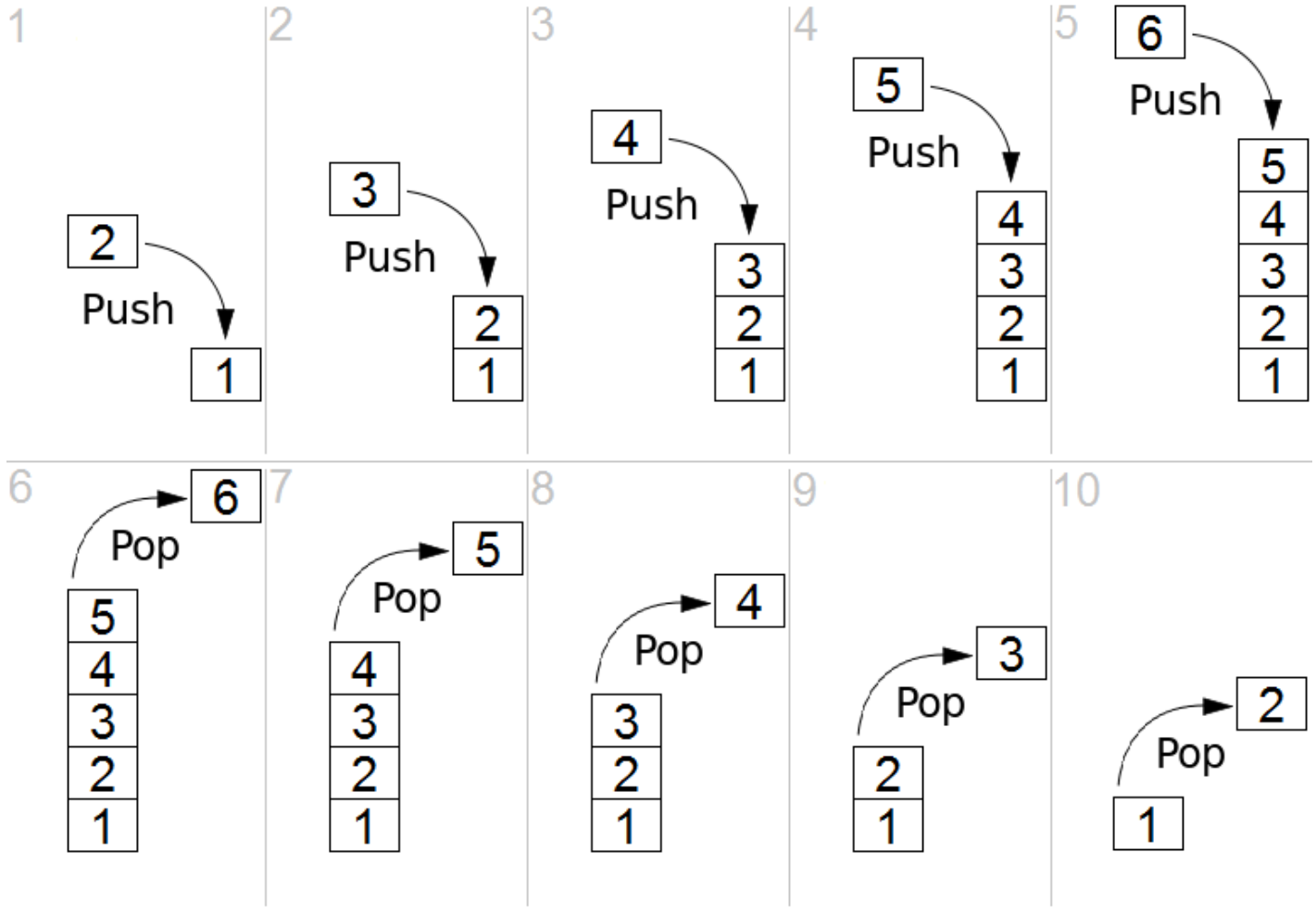
As $\mathcal{S}_{\emptyset}$ has no elements:

- `pop()` is not allowed
- `peek()` is not allowed.

The stack is a **LIFO** (**L**ast-**I**n, **F**irst-**O**ut) data structure, because:

- the last element that entered the stack (i.e., via `push()`)
- ...is the one that will be removed first (i.e., via `pop()`)

Effect of `push()` and `pop()` over a stack:



Simulating a Stack

You can simulate the behaviour of a stack with lists.

```
S = []          # start with an empty stack
S.insert(0, 1)  # step 1 in previous slide
S.insert(0, 2)  # step 2 in previous slide
S.insert(0, 3)  # step 3 in previous slide
S.insert(0, 4)  # step 4 in previous slide
S.insert(0, 5)  # step 5 in previous slide
S.insert(0, 6)  # step 6 in previous slide
print(S)
S.pop(0)        # step 7 in previous slide
S.pop(0)        # step 8 in previous slide
S.pop(0)        # step 9 in previous slide
S.pop(0)        # step 10 in previous slide
S.pop(0)        # step 11 in previous slide
print(S)
```

```
[6, 5, 4, 3, 2, 1]
[1]
```

RECALL: `lst.insert(x, y)` inserts value `y` at position `x` in list `lst`.

NOTE: Previously, we supposed that the head of the list is the top of the stack. You can also assume that the tail of the list is the head of the stack.

```
S = []      # start with an empty stack
S.append(1) # step 1 in slide 4
S.append(2) # step 2 in slide 4
S.append(3) # step 3 in slide 4
S.append(4) # step 4 in slide 4
S.append(5) # step 5 in slide 4
S.append(6) # step 6 in slide 4
print(S)
S.pop()     # step 7 in slide 4
S.pop()     # step 8 in slide 4
S.pop()     # step 9 in slide 4
S.pop()     # step 10 in slide 4
S.pop()     # step 11 in slide 4
print(S)
```

```
[1, 2, 3, 4, 5, 6]
[1]
```

Queues

The **queue** is another popular abstract data type in computer science.

A queue is a sequence (collection) of elements where only the "oldest" element can be accessed.

The allowed operations on a queue $\mathcal{Q}$ are the following:

- `enqueue(` e `,` $\mathcal{Q}$ `)` : adds a new element $e \in V$, for any domain V, at the beginning (back) of the queue $\mathcal{Q}$
- `dequeue(` $\mathcal{Q}$ `)` : returns and removes the current element at the end (front) of the queue $\mathcal{Q}$.

Optionally, it is possible to define an additional operation:

- `peek(` $\mathcal{Q}$ `)` : reads the last element without modifying the queue.

A special domain value is $\mathcal{Q}_\emptyset$, namely the empty queue.

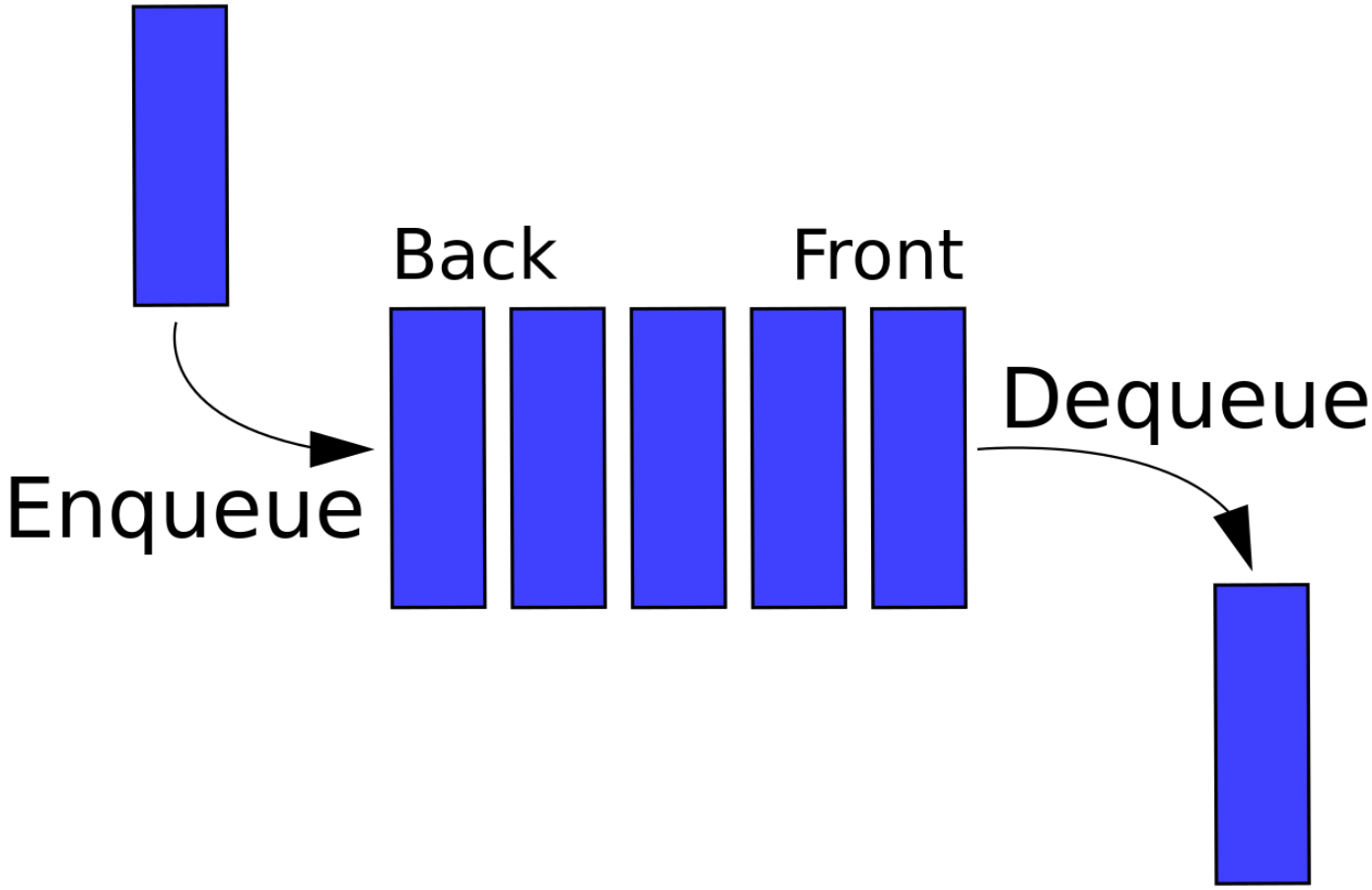
As $\mathcal{Q}_\emptyset$ has no elements:

- `dequeue()` is not allowed
- `peek()` is not allowed.

The queue is a FIFO (First-In, First-Out) data structure, because:

- the first element that entered the queue (i.e., via `enqueue()`)
- ...is the one that will be removed first (i.e., via `dequeue()`)

Visual representation of a queue:



Simulating a Queue

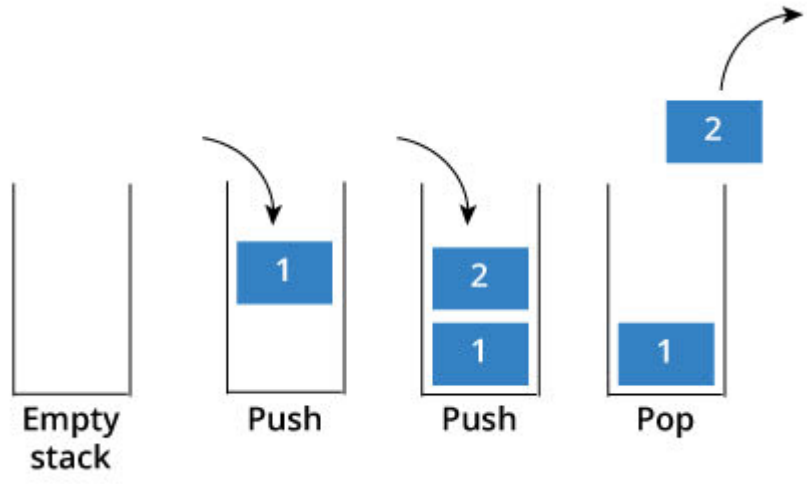
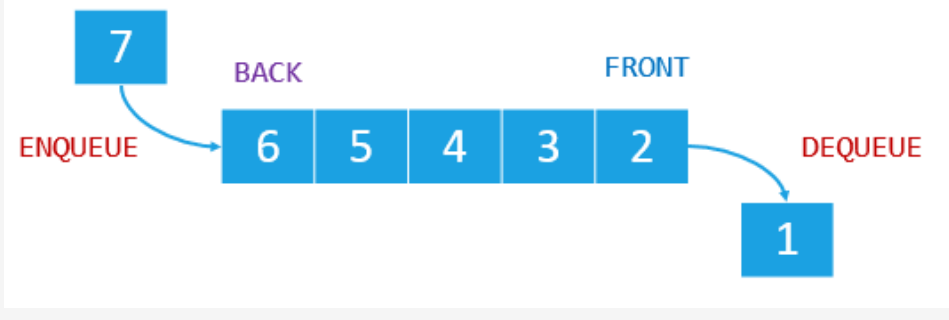
You can simulate the behaviour of a queue with lists.

```
Q = []           # start with an empty queue
Q.insert(0, 1)   # insert at the beginning = enqueue
Q.insert(0, 2)
Q.insert(0, 3)
Q.insert(0, 4)
Q.insert(0, 5)
Q.insert(0, 6)
print(Q)
Q.pop()          # pop at the end = dequeue
Q.pop()
Q.pop()
Q.pop()
Q.pop()
print(Q)
```

```
[6, 5, 4, 3, 2, 1]
[6]
```

NOTE: Here we suppose that the head of the list is the beginning (back) of the queue and that the tail of the list is the end (front) of the queue.

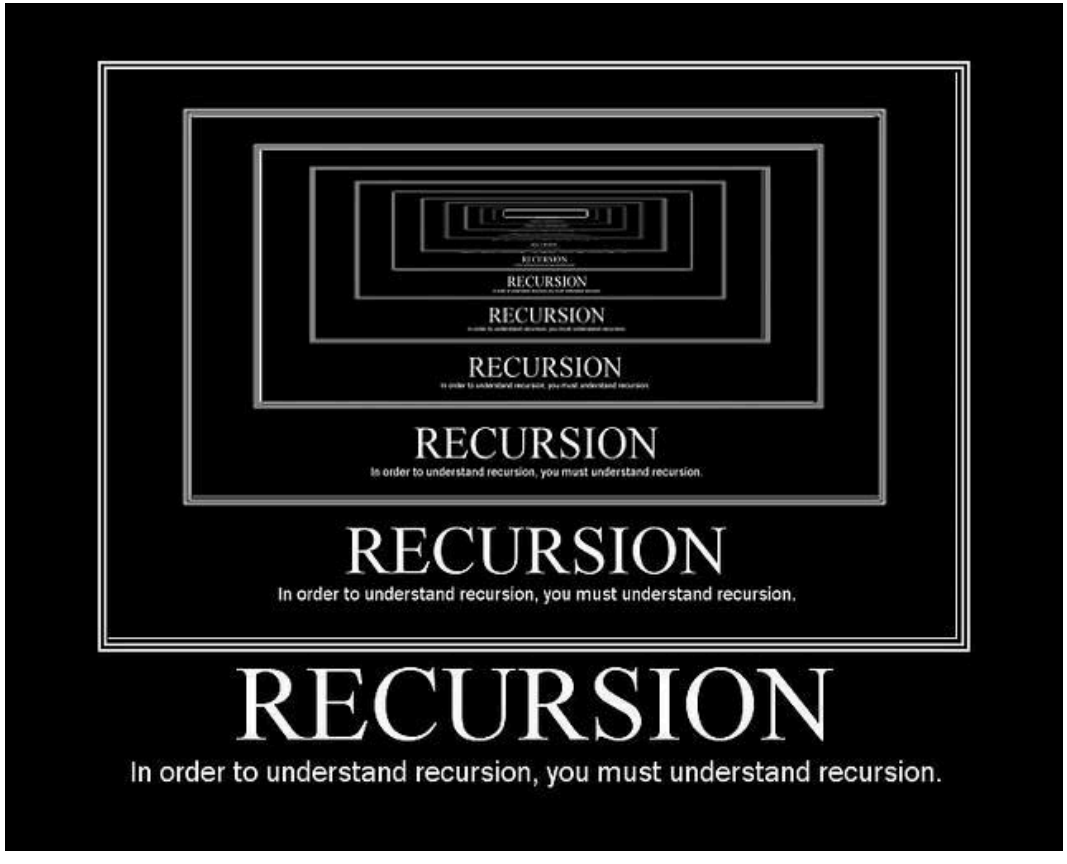
Queue vs. Stack

Data Structure	Visual Representation
Stack	
Queue	

Or, more informally:

Stack	Queue

Recursion


RECURSION
RECURSION
RECURSION
RECURSION
In order to understand recursion, you must understand recursion.
RECURSION
In order to understand recursion, you must understand recursion.
RECURSION
In order to understand recursion, you must understand recursion.

Recursion and Problem Solving

When designing the solution of a problem

...try to solve the problem assuming that you have *already solved it*!

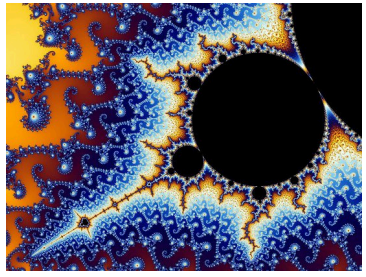
How is that possible?

Define the solution of a problem instance in terms of the solution of *smaller* problem instances.

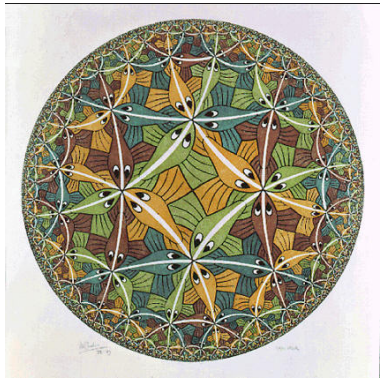
Practically speaking:

The function that computes the solution will *invoke itself* (once or more than once) from inside its body.

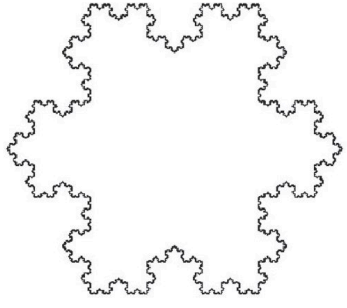
Real-world recursion



Mandelbrot Set



Escher's Circle Limit III



Koch Snowflake

Parallel Mirrors

A more practical example? Computing the factorial of a number n.

$$n! = n \cdot (n-1) \cdot (n-2) \cdot \ldots \cdot 1$$

Back-of-the-envelope calculation: $5!$

$$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot \underbrace{1}_{1!} = \underbrace{\underbrace{\underbrace{\underbrace{5 \cdot 4}_{5 \cdot 4!} \cdot 3}_{4 \cdot 3!} \cdot 2}_{3 \cdot 2!} \cdot 1}_{2 \cdot 1!}$$

We can spot the following pattern:

$$n! = n \cdot (n-1)!$$

with base case:

$$0! = 1$$

Recursion and Problem Solving (cont'd)

In order to use a recursive approach:

1. you must be able to decompose the problem into problems of the same type
2. you must find a relation that connects a given problem to the solution of similar subproblems
3. you need to know the solution of a special case of the problem (the base case, where the recursion terminates).

Take the factorial example:

1. multiplication between two numbers
2. multiplication between n and the already-known $(n-1)!$
3. the base case is $0! = 1$

Issues with Recursion

There are two major issues with recursion:

1. **Infinite recursion**
    - missing base case?
    - function called with wrong arguments? For example, what happens if we do `(-1)!`? ... more on this later
2. **Un-responsiveness**
    - your code can easily become very slow, also with "easy" input parameters
    - ... more on memory management later

NOTE: Python is quite good at catching infinite recursions: it has a dedicated error:
`RecursionError: maximum recursion depth exceeded in comparison`

Memory Management

Case Study: `countdown`

Write a recursive function `countdown(n)` that takes a positive integer `n` and prints the countdown. As the countdown reaches 0, prints `Blastoff!` and quits.

```
def countdown(n):
    if n==0:
        print("Blastoff!")
    else:
        print(n)
        countdown(n-1)

countdown(3)
```

```
3
2
1
Blastoff!
```

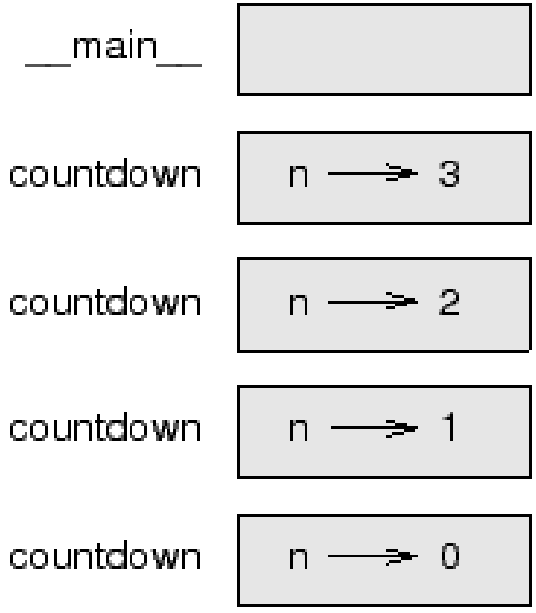
What's going on here?

- (1) The execution of `countdown` begins with `n=3`, and since `n` is greater than `0`, it outputs the value `3`, and then calls itself...
    - (2) The execution of `countdown` begins with `n=2`, and since `n` is greater than `0`, it outputs the value `2`, and then calls itself...
        - (3) The execution of `countdown` begins with `n=1`, and since `n` is greater than `0`, it outputs the value `1`, and then calls itself...
            - (4) The execution of `countdown` begins with `n=0`, and since `n` is not greater than `0`, it outputs `Blastoff!` and then returns.
        - (5) The countdown that got `n=1` returns.
    - (6) The countdown that got `n=2` returns.
- (7) The countdown that got `n=3` returns. And we are back into the `main` script.

RECALL: Every time a function gets called, Python creates a new function frame, which contains the function's local variables and parameters.

For a recursive function, there might be more than one frame at the same time. Python organises those frames as a stack.

For instance, the stack diagram for `countdown(3)` is:



How does the stack in Figure 10 works?

1. (as usual) at the top of the stack is the frame for the `main`*
2. we call `countdown(3)`, so there is a new frame in the stack
3. `countdown(3)` calls `countdown(2)`, so there is a new frame in the stack
4. `countdown(2)` calls `countdown(1)`, so there is a new frame in the stack
5. `countdown(1)` calls `countdown(0)`, so there is a new frame in the stack
6. `countdown(0)` does not make any further calls, so no more frames

We did a bunch of `push`es, now we must do some `pop`s:

1. `countdown(0)` finishes its execution (i.e., prints `Blastoff!` without doing anything else). Its frame is erased.
2. the control goes back to `countdown(1)`, which finishes its execution. Its frame is erased.
3. the control goes back to `countdown(2)`, which finishes its execution. Its frame is erased.
4. the control goes back to `countdown(3)`, which finishes its execution. Its frame is erased.
5. the control goes back to the `main`.

Exercises

`factorial`

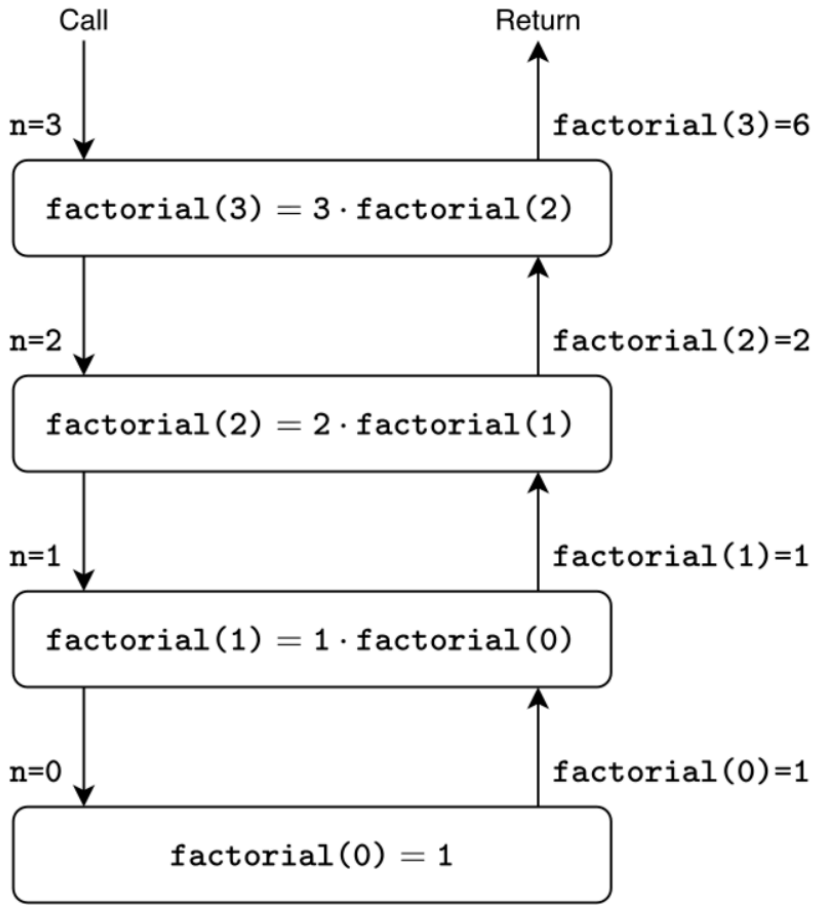
Write a recursive function `factorial(n)` that takes as input a positive integer `n` and returns $n!$

```
def factorial(n):
    if n==0:
        return 1
    else:
        return n * factorial(n-1)

output = factorial(5)
print(output)
```

```
120
```

WARNING: What happens when calling `factorial(-1)`?

Stack diagram for `factorial(3)`:



`palindrome`

Write a recursive function `palindrome(s)` that takes as input a string `s` and returns `True` if `s` is palindrome and `False` otherwise.

Decomposition strategy: let `s` be `"racecar"`

- racecar -> same letter ✓
- aceca -> same letter ✓
- cec -> same letter ✓
- e -> trivial ✓

and now let `s` be `"rediviuer"`

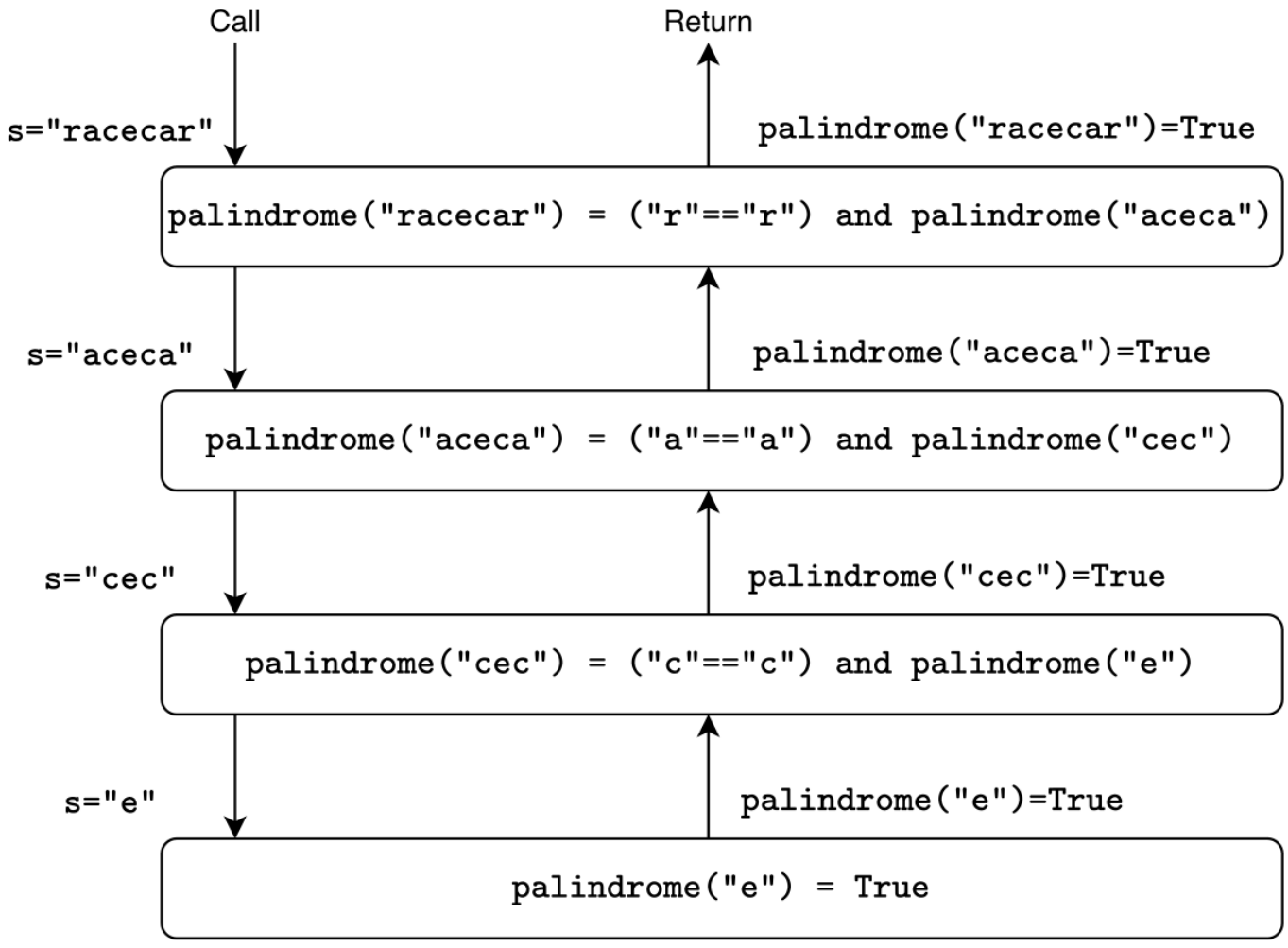
- rediviuer -> same letter ✓
- ediviue -> same letter ✓
- diviu -> different letter ✗

Hence, `"racecar"` is palindrome, but `"rediviuer"` it's not.

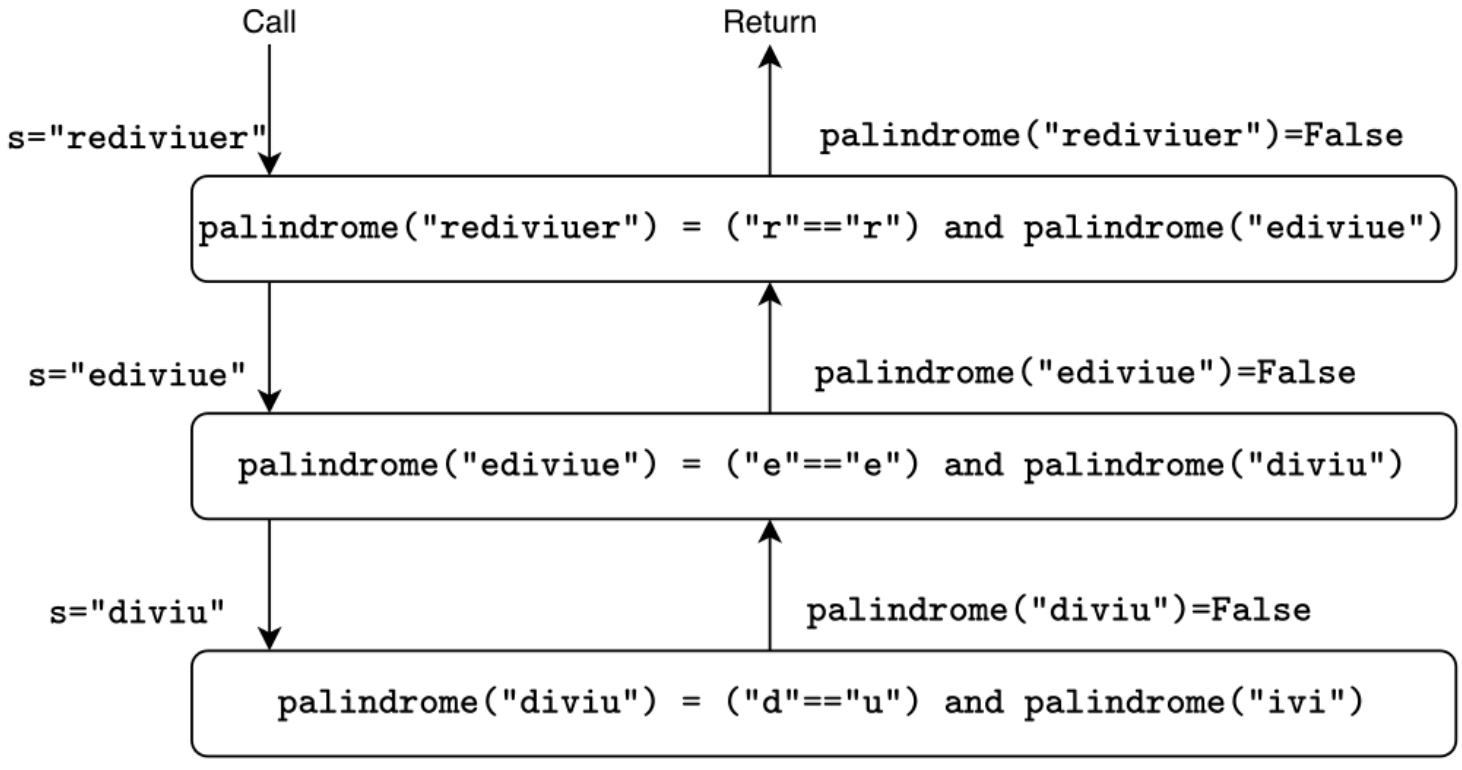
```
def palindrome(s):
    if len(s) <= 1:
        return True
    else:
        return (s[0] == s[-1]) and palindrome(s[1:-1])

output1 = palindrome("racecar")
print(output1)
output2 = palindrome("rediviuer")
print(output2)
```

```
True
False
```

Stack diagram for `palindrome("racecar")`:



Stack diagram for `palindrome("rediviuer")`:



`fibonacci`

Write a recursive function `fibonacci(n)` that takes as input a positive integer `n` and returns the n-th Fibonacci number.

RECALL: Each number in the Fibonacci sequence is given by the sum of the two preceding ones:

0 1 1 2 3 5 8 13 ...

Decomposition strategy: `fibonacci(n)` with `n=5`*:

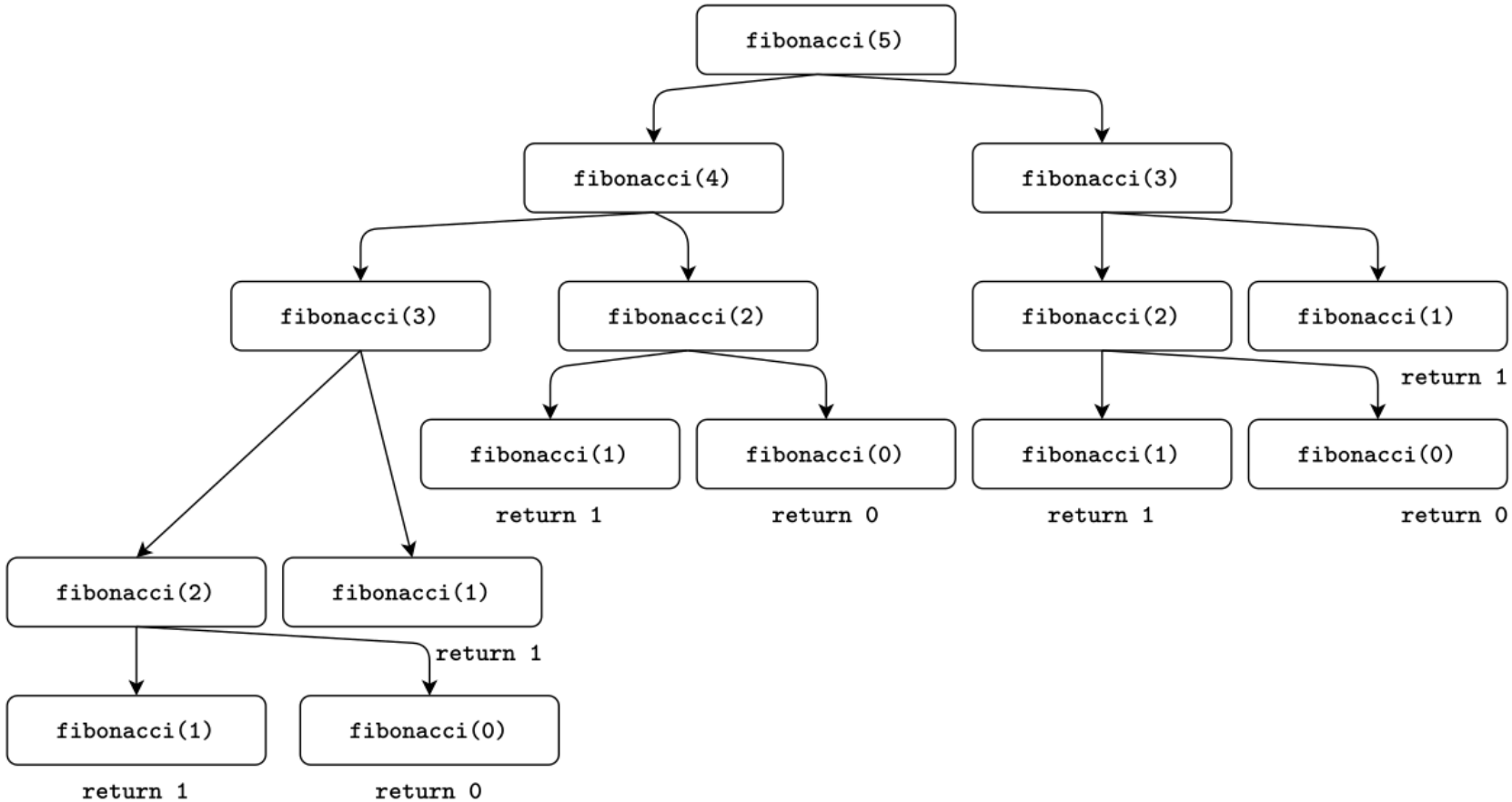
$$\overbrace{\overbrace{\overbrace{1+1}^{2}+1}^{3}+\overbrace{1+1}^{2}}^{5}$$

* By coincidence, the 5th Fibonacci number is 5.

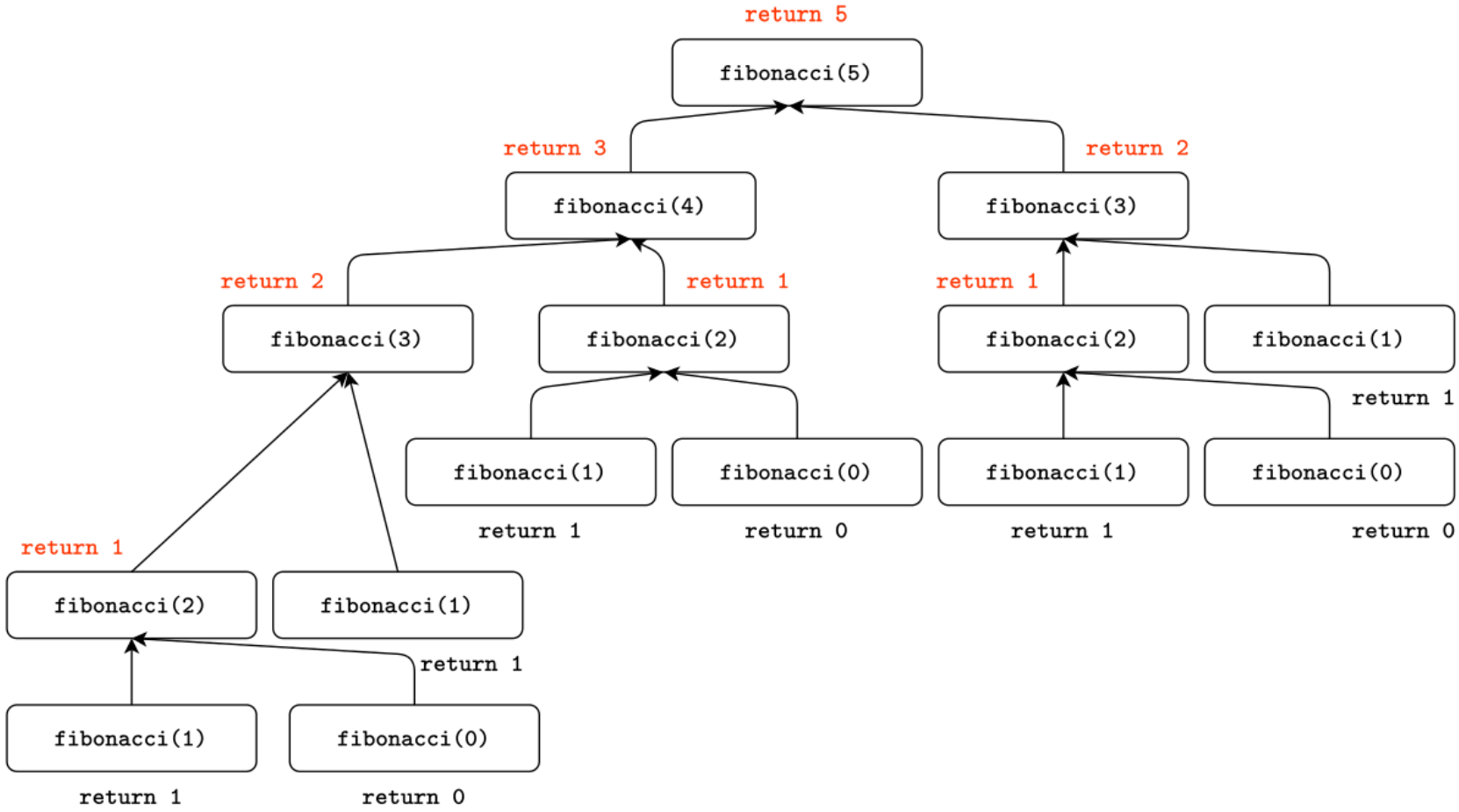
```
def fibonacci(n):
    if n in [0, 1]:
        return n
    else:
        return (fibonacci(n-1) + fibonacci(n-2))

output = fibonacci(5)
print(output)
```

```
5
```

Call Stack diagram for `fibonacci(5)`:



Return Stack diagram for `fibonacci(5)`:



Exercise session

Find the exercises for the exam session: here